

Reporte de la Actividad 5: Selección de la mediana

Equipo 9

Este documento analiza tres implementaciones para calcular la mediana de un arreglo desordenado de enteros sin recurrir a bibliotecas estándar de ordenamiento: ordenar y seleccionar, Quickselect con partición de Lomuto y selección determinista mediante mediana de medianas (BFPRT).

1. Descripción del problema y algoritmos

El problema corresponde a selección: dado un arreglo A de n elementos y un rango i , encontrar el i -ésimo elemento más pequeño. Para tamaños impares se devuelve el elemento de rango $(n+1)/2$; para tamaños pares, la mediana usual es el promedio de los dos elementos centrales. La actividad utiliza tamaños impares, de 100,001 a 1,000,001 elementos, cinco repeticiones por tamaño y reordenamiento aleatorio.

Ordena + selecciona. Ordena todo el arreglo y después accede a la posición central. Es directo y fácil de verificar, pero realiza trabajo adicional: produce un orden total aunque solo se necesita un elemento.

Quickselect con Lomuto. Escoge un pivote, recorre el segmento para colocar los menores a un lado y los mayores al otro, y continúa únicamente en la partición que contiene el rango buscado. Su ventaja es evitar la segunda rama de Quicksort; su debilidad es la sensibilidad a pivotes deficientes.

Mediana de medianas (BFPRT). Divide los elementos en grupos de cinco, calcula la mediana de cada grupo, selecciona recursivamente la mediana de esas medianas y la usa como pivote. Después particiona y continúa en una sola parte. El esquema está diseñado para garantizar un progreso suficiente incluso en el peor caso; las notas del MIT describen precisamente estos pasos. [web:6]

2. Complejidad del código y mantenimiento

Aquí “complejidad” se refiere a la dificultad de leer, modificar y verificar el código, no al tiempo de ejecución. La solución además integra una aplicación sencilla con Flask y HTML para introducir precios y simular casos, lo que separa la lógica algorítmica de la interfaz, pero añade rutas, plantillas y manejo de entradas.

Algoritmo	Tamaño del código	Mantenimiento
Ordena + selecciona	Bajo	Muy sencillo; pocas decisiones y fácil validación.
Lomuto	Medio	Requiere cuidar índices, límites, mutación y elección de pivote.
BFPRT	Alto	Incluye agrupación, medianas auxiliares, Es mas complicado.

La primera alternativa es la más mantenible porque delega la parte difícil en una operación conocida. Lomuto tiene una implementación compacta, pero los errores de índices pueden ser sutiles: un cambio en el pivote o en las condiciones de parada puede producir resultados incorrectos o degradación severa. BFPRT es conceptualmente más largo y exige documentar invariantes, tratamiento de grupos incompletos, duplicados y el rango relativo después de particionar. Como buena práctica, conviene mantener una función común para validar entradas, pruebas unitarias para arreglos pequeños y pruebas de regresión que comparen los tres resultados.

3. Análisis asintótico

Ordena + selecciona: el ordenamiento dominante cuesta $\Theta(n \log n)$ en los algoritmos de comparación con garantías habituales, y el acceso posterior cuesta $\Theta(1)$. Por tanto, el costo total es $\Theta(n \log n)$. El espacio depende de la implementación concreta y de si se copia el arreglo; una implementación que ordena una copia requiere $\Theta(n)$ espacio adicional.

Lomuto: cada partición examina $\Theta(m)$ elementos del segmento actual. Si los pivotes son razonablemente equilibrados, la suma de los tamaños examinados es $\Theta(n)$, por lo que el caso promedio esperado es $\Theta(n)$. Si el pivote queda repetidamente en un extremo, la recurrencia es $T(n)=T(n-1)+\Theta(n)$, que produce $\Theta(n^2)$. La versión in-place usa $\Theta(1)$ espacio auxiliar aparte de la pila recursiva; la profundidad puede ser $\Theta(\log n)$ en el caso

equilibrado y $\Theta(n)$ en el peor caso.

BFPRT: formar grupos y particionar cuesta $\Theta(n)$, la selección recursiva de las medianas cuesta $T(n/5)$, y el subproblema restante queda acotado por aproximadamente $7n/10$. La recurrencia $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$ se resuelve como $\Theta(n)$, tanto en promedio como en el peor caso. La garantía proviene de que el pivote descarta una fracción constante de los elementos; las notas del MIT presentan el algoritmo y la conclusión de tiempo lineal.

Algoritmo	Promedio	Peor caso	Espacio auxiliartípico
Ordena + selecciona	$\Theta(n \log n)$	$\Theta(n \log n)^*$	$\Theta(n)$ si copia
Lomuto	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$ + pila
BFPRT	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$ o menor según implementación

*La cota del primer método supone un ordenamiento de comparación eficiente; no debe confundirse con el costo de una biblioteca específica.

4. Benchmark y discusión

El protocolo indicado mide exclusivamente el cálculo, no la generación de datos ni el reordenamiento: arreglos impares entre 100,001 y 1,000,001 elementos, cinco repeticiones por tamaño y shuffle antes de cada prueba. Para reportar resultados reproducibles deben conservarse el lenguaje y versión, hardware, sistema operativo, semilla aleatoria, unidad de tiempo y si cada algoritmo recibe una copia idéntica del arreglo. El promedio debe acompañarse de dispersión, por ejemplo desviación estándar o intervalo mínimo–máximo.

El gráfico de tiempos debe incluir una serie por algoritmo, con el tamaño n en el eje horizontal y el tiempo promedio en el vertical. Como no se proporcionan en el material disponible los valores numéricos ni la imagen del benchmark, no es posible reconstruir honestamente el gráfico ni inventar sus mediciones. Debe insertarse aquí el gráfico generado por el programa de pruebas; si se observan valores atípicos, conviene mostrar también barras de error.

Teóricamente, ordenar y seleccionar debería crecer más rápido que los dos métodos lineales. Quickselect suele ser competitivo y, con datos aleatorios, puede superar a BFPRT por tener menos trabajo constante. BFPRT puede mostrar tiempos mayores para tamaños moderados debido a calcular medianas de grupos y realizar más pasos, aunque su crecimiento queda protegido frente a pivotes adversos. El benchmark debe contrastarse con estas tendencias, sin esperar que las curvas coincidan exactamente: asignaciones, copias, caché, intérprete, recursión, recolección de memoria y ruido del sistema afectan las constantes.

5. Conclusiones

La alternativa de ordenar es la más simple de mantener, pero no es la más eficiente asintóticamente para una sola mediana. Lomuto ofrece un equilibrio práctico entre código corto y costo lineal promedio, a cambio de un peor caso cuadrático. BFPRT proporciona la garantía teórica más sólida, $\Theta(n)$ en el peor caso, pero exige mayor cuidado de implementación y documentación. La elección final depende de si se prioriza simplicidad, rendimiento práctico o una garantía determinista.

Link a Github

<https://github.com/ppgrio/TC2038.602-Equipo-9>